DATE: 22/10/2026

CO2-AT1- ANALYTICAL PROBLEM SOLVING

CODE: DSA0308

COURSE NAME: NATURAL LANGUAGE PROCESSING

REG NO: 192324281 **NAME:** MONIKA R

ANSWERS

QUESTION 1: MORPHOLOGICAL ANALYSIS

PROBLEM STATEMENT

Design a morphological analysis pipeline for the words “connected”, “connecting”, and “connection”. Identify their roots and suffixes, classify the forms as inflectional or derivational, and normalize them to a common base form for document indexing.

ANALYTICAL APPROACH

- Connected = connect + -ed. The suffix indicates past tense and is inflectional.
- Connecting = connect + -ing. The suffix indicates an ongoing action and is inflectional.
- Connection = connect + -ion. The suffix forms a noun and is derivational.

All three words are normalized to connect.

ALGORITHM

1. Store the input words in a list.
2. Identify the root and suffix of each word.
3. Classify each suffix as inflectional or derivational.
4. Normalize all words to the common root connect.
5. Display the original word, root, suffix, classification, and normalized form.

INTERMEDIATE STEPS

Original word	Root	Suffix	Classification	Normalized form
connected	connect	-ed	Inflectional	connect
connecting	connect	-ing	Inflectional	connect
connection	connect	-ion	Derivational	connect

Explanation:

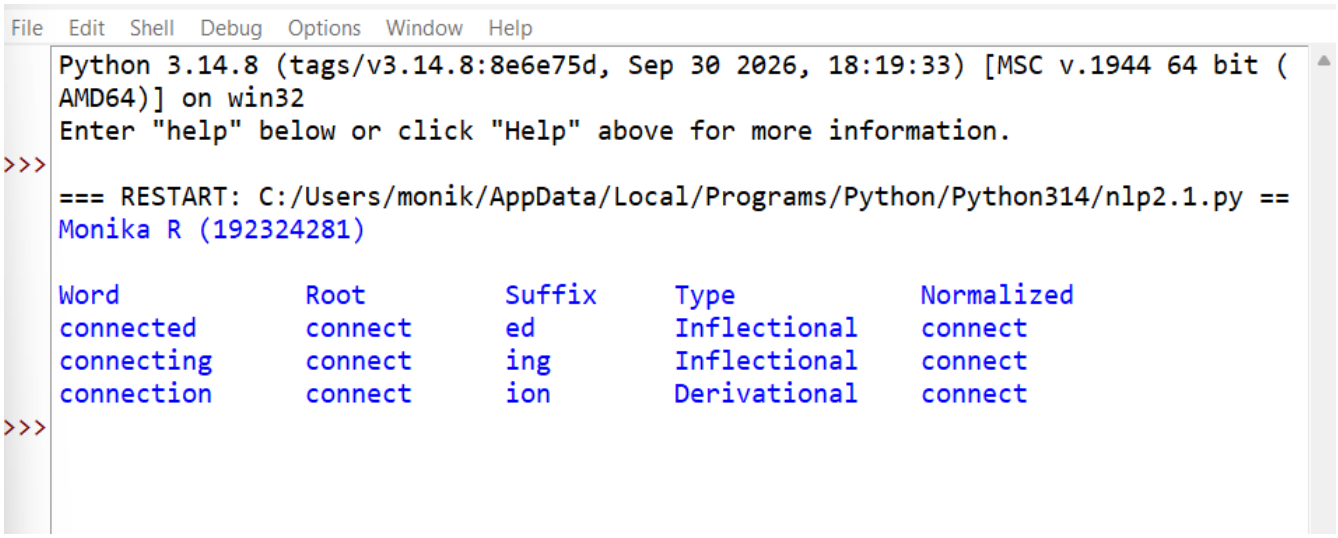
- connect + ed → connected: -ed indicates past tense.
- connect + ing → connecting: -ing indicates an ongoing action.
- connect + ion → connection: -ion forms a noun from the verb.

The first two transformations are inflectional because they change the grammatical form. The third is derivational because it forms a new word class.

CODE

```
print("Monika R (192324281)")
words = ["connected", "connecting", "connection"]
print("\n{:<15} {:<12} {:<10} {:<15} {}".format(
    "Word", "Root", "Suffix", "Type", "Normalized"
))
for word in words:
    if word == "connected":
        root = "connect"
        suffix = "ed"
        kind = "Inflectional"
    elif word == "connecting":
        root = "connect"
        suffix = "ing"
        kind = "Inflectional"
    else:
        root = "connect"
        suffix = "ion"
        kind = "Derivational"
    print("{:<15} {:<12} {:<10} {:<15} {}".format(
        word, root, suffix, kind, root
    ))
```

OUTPUT



RESULT

Thus, the morphological analysis pipeline identifies the root and suffix of each input word, classifies the transformations as inflectional or derivational, and normalizes all three words to connect for efficient document indexing and information retrieval.

QUESTION 2: MORPHOLOGICAL PARSING FOR SENTIMENT ANALYSIS

PROBLEM EXPLANATION

Design a morphological parsing module for the words “unhappy”, “happiness”, and “happily”. Identify their prefixes, suffixes, and base forms, classify the transformations, and normalize them for sentiment analysis.

ANALYTICAL APPROACH

- Unhappy = un- + happy. The prefix un- indicates the opposite meaning. It is derivational.
- Happiness = happy + -ness. The suffix -ness forms a noun. It is derivational.
- Happily = happy + -ly. The suffix -ly forms an adverb. It is derivational.

All three words are normalized to the root happy.

INTERMEDIATE STEPS

Original word	Prefix	Suffix	Base form	Classification	Normalized root
unhappy	un-	None	happy	Derivational	happy
happiness	None	-ness	happy	Derivational	happy
happily	None	-ly	happy	Derivational	happy

ALGORITHM

1. Store the input words in a list.
2. Identify the prefix, suffix, and base form of each word using rules.
3. Classify each transformation as inflectional or derivational.
4. Identify the semantic effect of each affix.
5. Normalize all words to the base form “happy”.
6. Display the morphological breakdown and normalized root in tabular format.

PYTHON PROGRAM

```
print("Monika R (192324281)")

words = ["unhappy", "happiness", "happily"]

print("\n{:<12} {:<10} {:<10} {:<10} {:<15} {}".format(
    "Word", "Prefix", "Suffix", "Base", "Type", "Root"
```

```
))
for word in words:
    if word == "unhappy":
        prefix = "un-"
        suffix = "None"
        base = "happy"
        kind = "Derivational"
    elif word == "happiness":
        prefix = "None"
        suffix = "-ness"
        base = "happy"
        kind = "Derivational"
    else:
        prefix = "None"
        suffix = "-ly"
        base = "happy"
        kind = "Derivational"
    root = "happy"
    print("{:<12} {:<10} {:<10} {:<10} {:<15} {}".format(
        word, prefix, suffix, base, kind, root
    ))
```

OUTPUT

```
Python 3.14.8 (tags/v3.14.8:8e6e75d, Sep 30 2026, 18:19:33) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/monik/AppData/Local/Programs/Python/Python314/nlp2.1.py ==
Monika R (192324281)

Word         Prefix     Suffix     Base       Type            Root
unhappy      un-        None       happy      Derivational    happy
happiness    None       -ness      happy      Derivational    happy
happily      None       -ly        happy      Derivational    happy
>>>
```

8.

RESULT

The program identifies the morphological components and normalizes all three words to happy. The negative meaning of “unhappy” should be preserved during sentiment analysis.

QUESTION 3: STEMMING-BASED PREPROCESSING FOR A SEARCH ENGINE

PROBLEM EXPLANATION

Design a stemming-based preprocessing module for the words “played”, “player”, and “playing”. Identify their stems, remove suffixes, classify transformations as inflectional or derivational, and normalize the words for efficient search and document retrieval.

ANALYTICAL APPROACH

- Played = play + -ed. The suffix -ed indicates past tense. It is inflectional.
- Player = play + -er. The suffix -er forms a noun referring to a person. It is derivational.
- Playing = play + -ing. The suffix -ing indicates an ongoing action. It is inflectional.

All three words are normalized to the stem play.

INTERMEDIATE STEPS

Original word	Stem	Removed affix	Classification	Normalized form
played	play	-ed	Inflectional	play
player	play	-er	Derivational	play
playing	play	-ing	Inflectional	play

ALGORITHM

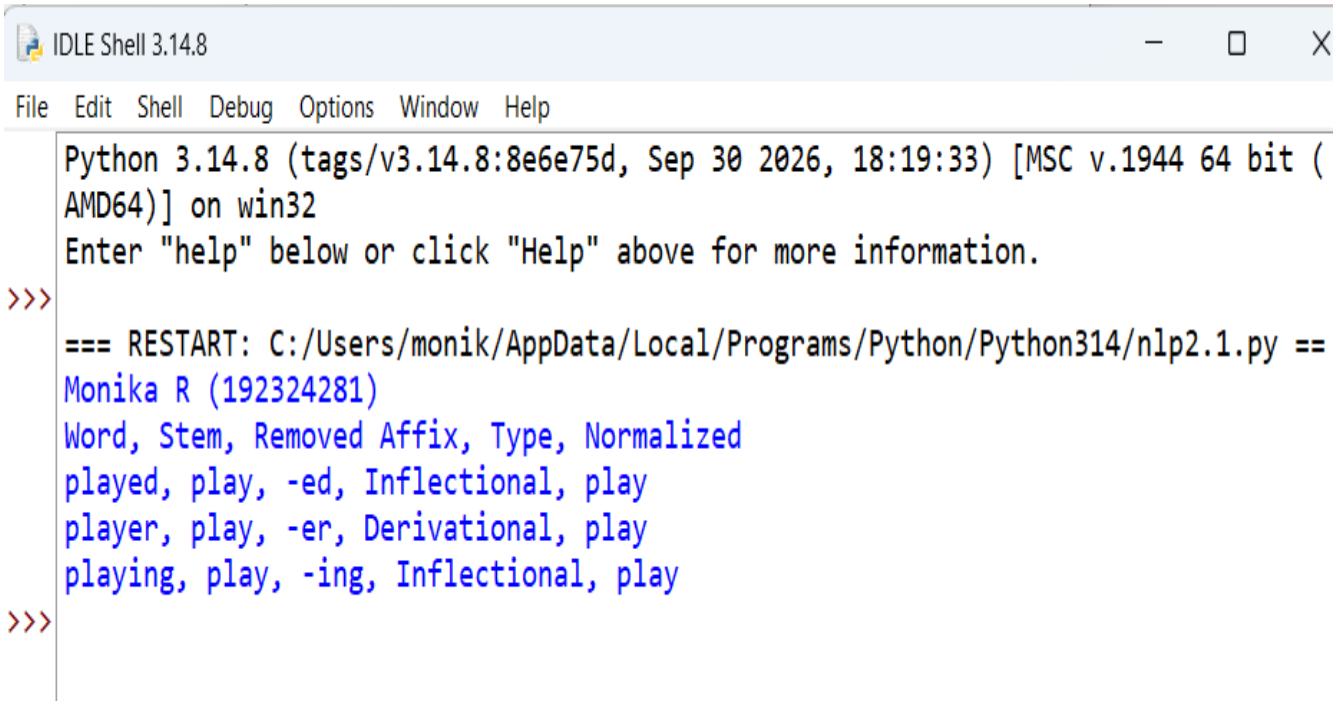
1. Store the input words.
2. Identify the stem and suffix of each word.
3. Classify each transformation.
4. Normalize all words to play.
5. Display the results in tabular format.

PYTHON PROGRAM

```
print("Monika R (192324281)")
words = ["played", "player", "playing"]
print("Word, Stem, Removed Affix, Type, Normalized")
for word in words:
    if word == "played":
        affix = "-ed"
```

```
    kind = "Inflectional"
elif word == "player":
    affix = "-er"
    kind = "Derivational"
else:
    affix = "-ing"
    kind = "Inflectional"
print(word, "play", affix, kind, "play", sep=", ")
```

OUTPUT



RESULT

The program identifies play as the common stem of all three words and classifies their transformations. This normalization improves search efficiency by matching related word forms.

QUESTION 4: FINITE-STATE MORPHOLOGICAL PARSING

PROBLEM EXPLANATION

Design a finite-state morphological parsing module for the words “writes”, “writing”, and “written”. Identify their root forms, model regular and irregular verb inflections, trace state transitions, and normalize all words to a common root.

ANALYTICAL APPROACH

- Writes: write + -s. The suffix -s indicates third-person singular present tense. It is a regular inflection.
- Writing: write + -ing. The suffix -ing indicates an ongoing action. It is a regular inflection.
- Written: write + -en. This is an irregular past participle form.

All three words are normalized to the root write.

INTERMEDIATE STEPS

Original word	Root	Morphological component	Classification	State transition	Normalized form
writes	write	-s	Regular	START → ROOT → INFLECTION → FINAL	write
writing	write	-ing	Regular	START → ROOT → INFLECTION → FINAL	write
written	write	-en	Irregular	START → ROOT → IRREGULAR → FINAL	write

ALGORITHM

1. Start the program.
2. Store the input words and their root forms.
3. Define states for root identification and inflection processing.
4. Apply regular suffix rules and an irregular-form mapping.
5. Trace the state transitions for each word.
6. Display the root, classification, transition path, and normalized form.
7. Stop the program.

PYTHON PROGRAM

```
print("Monika R (192324281)")

words = {

   "writes": ("write", "-s", "Regular"),

   "writing": ("write", "-ing", "Regular"),
```

```
    "written": ("write", "-en", "Irregular")
}
for word, data in words.items():
    root, suffix, kind = data
    if kind == "Regular":
        path = "START -> ROOT -> INFLECTION -> FINAL"
    else:
        path = "START -> ROOT -> IRREGULAR -> FINAL"
    print("\nWord:", word)
    print("Root:", root)
    print("Suffix:", suffix)
    print("Type:", kind)
    print("State Path:", path)
    print("Normalized Form:", root)
```

OUTPUT

```
=== RESTART: C:/Users/monik/AppData/Local/Programs/Python/Python314/nlp2.1.py ==
Monika R (192324281)

Word: writes
Root: write
Suffix: -s
Type: Regular
State Path: START -> ROOT -> INFLECTION -> FINAL
Normalized Form: write

Word: writing
Root: write
Suffix: -ing
Type: Regular
State Path: START -> ROOT -> INFLECTION -> FINAL
Normalized Form: write

Word: written
Root: write
Suffix: -en
Type: Irregular
State Path: START -> ROOT -> IRREGULAR -> FINAL
Normalized Form: write
>>>
```

RESULT

The finite-state parsing module identifies write as the common root of all three words, distinguishes regular inflections from the irregular form written, and normalizes the words for text processing and information retrieval.

QUESTION 5: PORTER STEMMER-BASED PREPROCESSING

PROBLEM EXPLANATION

Design a Porter Stemmer-based preprocessing module for the words “relational”, “relation”, and “relate”. Apply stemming rules to remove suffixes, identify intermediate forms, and generate final stems for document retrieval.

ANALYTICAL APPROACH

- Relational: The suffix -ational is reduced to -ate, producing relate. The final e is removed to obtain relat.
- Relation: The suffix -ion is removed under the Porter stemming rules, producing relat.
- Relate: The final suffix -e is removed, producing relat.

All three words are normalized to the common stem relat.

INTERMEDIATE STEPS

Original word	Applied rule	Intermediate form	Final stem
relational	-ational → -ate	relate	relat
relation	Remove -ion	relat	relat
relate	Remove final -e	relat	relat

ALGORITHM

1. Start the program.
2. Import the Porter Stemmer from NLTK.
3. Store the input words in a list.
4. Apply Porter stemming rules to each word.
5. Display the original word, intermediate form, and final stem.
6. Stop the program.

PYTHON PROGRAM

```
print("Monika R (192324281)")

from nltk.stem import PorterStemmer

ps = PorterStemmer()

words = ["relational", "relation", "relate"]

for word in words:

    stem = ps.stem(word)

    print("\nOriginal Word:", word)

    print("Final Stem:", stem)

    print("Normalized Form:", stem)
```

OUTPUT

```
>>>
    === RESTART: C:/Users/monik/AppData/Local/Programs/Python/Python314/nlp2.1.py ==
    Monika R (192324281)

    Original Word: relational
    Final Stem: relat
    Normalized Form: relat

    Original Word: relation
    Final Stem: relat
    Normalized Form: relat

    Original Word: relate
    Final Stem: relat
    Normalized Form: relat
```

RESULT

The Porter Stemmer successfully reduces relational, relation, and relate to the common stem relat. This normalization helps improve document indexing and information retrieval by matching related word forms.